FINAL REPORT

Capstone Project in Data Science II (DS4105)

Sentiment Analysis of McDonald's Customer Reviews: A Comparative Study of Classical Machine Learning, Transformer and Lexicon-Based Methods with a Deployed Intelligence Platform



Submitted by

Pooja Anbalagan

Index No: 22CDS0412

Internal Supervisor: **Ms. Sweshthika Sureshkumar**

DEPARTMENT OF DATA SCIENCE

FACULTY OF COMPUTING

SABARAGAMUWA UNIVERSITY OF SRI LANKA

September, 2026

Declaration

I declare that this report does not incorporate, without acknowledgement, any material previously submitted for a Degree or a Diploma in any University, and that, to the best of my knowledge and belief, it does not contain any material previously published or written by another person or myself except where due reference is made in the text. Also, I hereby grant Sabaragamuwa University of Sri Lanka the non-exclusive right to reproduce and distribute my report, in whole or in part, in print, electronic, or other media. I retain the right to use this content in whole or in part in future works (such as articles or books).

Index Number	: 22CDS0412
Name of Student	: Pooja Anbalagan
Date	: 28 September 2026
Signature of Student	: .....

Certificate of Approval

We hereby declare that this report is the student's own work and effort, and that all other sources of information used have been acknowledged. This report has been submitted with our approval.

Name of Internal Supervisor	: Ms. Sweshthika Sureshkumar
Date	: 28 September 2026
Signature of Internal Supervisor	: .....
Name of Head of the Department	: Dr UAP Ishanka
Date	: 28 September 2026
Signature of Head of the Department	: .....

Acknowledgement

I would like to express my sincere gratitude to my supervisor, Ms. Sweshthika Sureshkumar, for the invaluable guidance and constructive feedback provided throughout the course of this Capstone Project. Her expertise in data science and machine learning greatly shaped the direction and quality of this work.

I am also thankful to the Head of the Department of Data Science and the academic staff of the Faculty of Computing, Sabaragamuwa University of Sri Lanka, for providing the resources and academic environment necessary to complete this project. I extend my appreciation to the annotators who contributed to the labelling pilot, and finally to my friends and colleagues for their unwavering support and encouragement during this endeavour.

Abstract

Customer reviews on public platforms are a rich but unstructured source of feedback for multi-outlet service businesses, and manual analysis does not scale. This project builds and evaluates an end-to-end sentiment analysis system for 33,396 McDonald's Google Maps reviews. After removing encoding artefacts, 10,741 duplicates and 179 unusable rows, and isolating Google's auto-generated rating tags to prevent label leakage, the pipeline retains 22,448 free-text reviews labelled Negative, Neutral or Positive through a hybrid scheme of 1,000 human-annotated reviews (Fleiss' kappa 0.6287) and 21,448 model pseudo-labels. Using a leakage-free 70/15/15 stratified split, the study benchmarks five TF-IDF classifiers, the VADER lexicon baseline and a fine-tuned DistilBERT transformer, with macro-F1 as the primary metric because the Neutral class forms only 15.73% of the data. DistilBERT achieves the best results (accuracy 0.909, macro-F1 0.891, Neutral F1 0.814), followed by Logistic Regression and Linear SVM, which are indistinguishable within cross-validation variability (macro-F1 0.829 for both). Multinomial Naive Bayes reaches 0.775 accuracy but a macro-F1 of only 0.586 because it almost never predicts Neutral, and VADER reaches a macro-F1 of 0.554. Roughly three-quarters or more of the errors of the linear models involve the Neutral class. Predictive-term analysis shows that slow service, rudeness and dirt drive negative sentiment, whereas friendliness and speed drive positive sentiment. The models are served through a FastAPI service and a React dashboard offering live prediction, batch scoring, analytics and report export. The results show that fine-tuned transformers deliver a clear gain on ambiguous reviews, while linear models remain a fast, interpretable alternative for low-latency use.

Keywords: sentiment analysis; natural language processing; TF-IDF; DistilBERT; text classification; VADER; pseudo-labelling; customer reviews

Table of Contents

Declaration 2
Certificate of Approval 3
Acknowledgement 4
Abstract 5
Table of Contents 6
List of Figures 9
List of Tables 10
Chapter 1: Introduction 11
1.1 Overview 11
1.2 Motivation 11
1.3 Major Goals and Objectives 12
1.4 Scope of the Completed Project 12
1.5 Approach and Assumptions 12
1.6 Summary of Major Outcomes 13
1.7 Organisation of the Report 13
Chapter 2: Background 14
2.1 Sentiment Analysis and Opinion Mining 14
2.2 The Problem Addressed 14
2.3 Related Work 14
2.3.1 Lexicon-based methods 14
2.3.2 Classical supervised learning 14
2.3.3 Transformer-based models 15
2.3.4 Weak supervision and pseudo-labelling 15
2.3.5 Imbalanced classification 15
2.4 Gap and Contribution 15
Chapter 3: Specification and Design 16
3.1 Users and Stakeholders 16
3.2 Functional Requirements 16
3.3 Non-Functional Requirements 17

3.4 Use Case Diagram 17
3.5 System Architecture 18
3.6 Sequence of a Prediction Request 20
Chapter 4: Methodology 21
4.1 Data Collection 22
4.2 Data Cleaning and Preprocessing 23
4.2.1 Auto-generated rating tags and label leakage 23
4.3 Sentiment Labelling 24
4.4 Exploratory Data Analysis 25
4.5 Text Preprocessing and Feature Engineering 27
4.6 Data Splitting and Cross-Validation 28
4.7 Model Selection, Training and Testing 29
4.8 Evaluation Metrics 30
4.9 Software and Hardware Requirements 31
4.10 Model Integration and Deployment 31
4.11 Difficulties Encountered and Solutions 32
Chapter 5: Results and Evaluation 33
5.1 Overall Performance 33
5.2 Per-Class Performance 34
5.3 Stability under Cross-Validation 34
5.4 Probabilistic Quality 35
5.5 Confusion Matrices and Error Analysis 35
5.6 Most Predictive Terms 36
5.7 Lexicon Baseline (VADER) 37
5.8 Efficiency 38
5.9 Robustness, Ablation and Error Diagnostics 38
5.10 Limitations and Threats to Validity 40
5.11 Discussion 40
Chapter 6: Future Work 41
6.1 Gaps in the Present Work 41
6.2 Proposals for Enhancement 41
Chapter 7: Conclusion 42

References 43
Appendix A: Repository and Artefact Structure 45
Appendix B: Reproduction Commands and Result Files 46
Glossary 47

List of Figures

Figure 1 - Use case diagram of the Sentiment Intelligence Platform. 18
Figure 2 - System Architecture of Sentiment Intelligence Platform. 19
Figure 3 - Sequence diagram of the Sentiment Intelligence Platform. 20
Figure 5 - Overview of the experimental and deployment pipeline 21
Figure 6 - Free-text versus auto-generated reviews after auto-tag isolation..... 24
Figure 7 : Sentiment class distribution (left) and star-rating distribution (right)..... 25
Figure 8 - Distribution of review length (characters) by sentiment class 26
Figure 9 - Sentiment composition by review recency 26
Figure 10 - Word clouds by sentiment class 27
Figure 11 - Most frequent bigrams by sentiment class 27
Figure 12 - Accuracy, macro precision, macro recall and macro-F1 of all models on the untouched test set. 34
Figure 13 - Confusion matrices of Logistic Regression (left) and Linear SVM (right) on the test set..... 36
Figure 14 : Top predictive terms per sentiment class from the linear model coefficients..... 37
Figure 15 - VADER compound sentiment score by star rating..... 37
Figure 16 - Confusion matrix of the VADER baseline (rows: actual class; columns: predicted class). 38

List of Tables

Table 1 - Functional requirements of the platform 16
Table 2 - Non-functional requirements of the platform 17
Table 3 - Principal Fields of the cleaned and labelled review corpora 22
Table 4 - The ten-stage data cleaning and preprocessing flow 23
Table 5 - Final label distribution and comparison with the star-rating mapping. 25
Table 6 -TF-IDF feature configuration 28
Table 7 - Model suite and configuration 29
Table 8 - DistilBERT fine-tuning settings 29
Table 9 - Evaluation metrics 30
Table 10 - Software and hardware environment 31
Table 11 - Difficulties encountered during implementation and how they were resolved 32
Table 12 - Test-set performance of all models 33
Table 13 - F1-score per class on the test set 34
Table 14 - Five-fold cross-validated macro-F1 (mean ± standard deviation) 35
Table 15 - Probability-based metrics on the test set 35
Table 16 - Per-class precision, recall and F1 for the linear models, computed from the confusion matrices. 36
Table 17 - Summary of robustness and ablation results 39

Chapter 1: Introduction

1.1 Overview

Customer reviews posted on public platforms such as Google Maps have become one of the most direct and abundant channels through which consumers express satisfaction or dissatisfaction with a service. For a high-volume, multi-outlet business such as McDonald's, thousands of reviews accumulate continuously, each combining a numeric star rating with free-text commentary. Reading and categorising this volume of feedback by hand is impractical, which motivates automated sentiment analysis: the task of algorithmically classifying a piece of text as expressing a Positive, Neutral or Negative opinion [1], [2].

This report presents the Capstone Project in Data Science II undertaken at the Department of Data Science, Faculty of Computing, Sabaragamuwa University of Sri Lanka. The project builds a complete sentiment analysis system for 33,396 McDonald's reviews collected from Google Maps. It covers data auditing and cleaning, a hybrid human and model-assisted labelling protocol, exploratory analysis, leakage-free experimental design, the benchmarking of seven sentiment-scoring approaches (five classical TF-IDF classifiers, a lexicon baseline and a fine-tuned transformer), error and robustness analysis, and deployment as an interactive web platform, the McDonald's Review Sentiment Intelligence Platform.

1.2 Motivation

Review sentiment analysis sits at the intersection of natural language processing and practical business analytics. It requires the full data science lifecycle, from messy scraped data through to a deployed, user-facing system, and it exposes several problems that are easy to overlook in textbook settings. Three motivations shaped this project.

- **Operational value.** For a fast-food operator, complaints about drive-thru delays, order accuracy or cleanliness are directly actionable. Surfacing them automatically and continuously from free text is far faster than manual reading.
- **Data quality.** Real scraped data contains encoding errors, duplicate records and, as this project shows, auto-generated placeholder text that mirrors the star rating and can silently leak the label into a model.
- **Evaluation honesty.** Sentiment corpora are class-imbalanced and the Neutral class is intrinsically ambiguous. Reporting accuracy alone can hide a classifier that never predicts Neutral, so the project adopts macro-F1 as its primary criterion and tests every claim on data that no model has seen.

1.3 Major Goals and Objectives

The overall goal is to design, evaluate and deploy a reliable three-class sentiment classifier for McDonald's customer reviews and to translate its outputs into insight that operational teams can use. The specific objectives are as follows.

1. Acquire, audit and clean a large real-world review corpus, resolving encoding artefacts, duplicates and label-leaking auto-generated tags.
2. Construct trustworthy sentiment labels through a hybrid protocol that combines a multi-annotator human pilot with model-generated pseudo-labels, and quantify annotator agreement.
3. Explore the corpus through the distribution, length, recency and lexical profile of each sentiment class.
4. Build a zero-leakage experimental design with a stratified 70/15/15 split, in which the test set is used only once for final evaluation.
5. Train and compare Logistic Regression, Linear SVM, Multinomial Naive Bayes, SGD and Ridge classifiers on TF-IDF features, together with a VADER lexicon baseline and a fine-tuned DistilBERT transformer.
6. Evaluate the models with imbalance-aware and probabilistic metrics, analyse their errors, and assess robustness and inference efficiency.
7. Interpret the linguistic drivers of each class and deploy the models through a FastAPI service and a React dashboard for live, batch and analytical use.

1.4 Scope of the Completed Project

The project analyses English-language Google Maps reviews of McDonald's outlets. It delivers a reproducible set of Jupyter notebooks and modular Python source code, serialised classical models and a fine-tuned DistilBERT model, an automated evaluation pipeline, a labelling workflow with documented decisions, a web platform, technical audit and methodology reports, and this final report. Aspect-level sentiment (for example, separating opinions about food from opinions about service within one review), multilingual reviews and other business domains are outside the scope and are discussed as future work in Chapter 6.

1.5 Approach and Assumptions

The project follows a standard applied data science workflow, from data acquisition to deployment, with each stage implemented in numbered notebooks and reusable modules under the src directory. The following assumptions underpin the work.

- Review text is the primary carrier of sentiment. Star ratings, store addresses and timestamps are deliberately excluded from the text feature matrices to avoid leakage.

- The three sentiment classes (Negative, Neutral, Positive) are adequate for the business question, and the Neutral class represents mixed, indifferent or borderline opinion.
- Human annotation is the reference standard for sentiment but is too costly to apply to every review. A 1,000-review human sample, with quantified agreement, is therefore used to anchor labels, and the remainder is labelled by a model (pseudo-labelling) [15].
- Because the Neutral class is a minority (15.73% of the corpus), macro-averaged metrics are more informative than accuracy for comparing models [16].

1.6 Summary of Major Outcomes

After cleaning, 22,448 free-text reviews remain, of which 47.34% are Negative, 36.94% are Positive and 15.73% are Neutral. Three annotators labelling a 1,000-review pilot reached a Fleiss' kappa of 0.6287, which corresponds to substantial agreement on the Landis and Koch scale [14], [17].

On the untouched test set, the fine-tuned DistilBERT model is the strongest classifier, with an accuracy of 0.9093 and a macro-F1 of 0.8906. Its Neutral-class F1 of 0.8139 is 0.132 higher than that of Logistic Regression. Logistic Regression and Linear SVM form a strong classical tier (macro-F1 0.8233 and 0.8195, and identical cross-validated means of 0.8294) with sub-millisecond inference, whereas Multinomial Naive Bayes reaches 0.7749 accuracy but only 0.5863 macro-F1 because it almost never predicts Neutral. The VADER lexicon baseline reaches an accuracy of 0.6171. Between roughly three-quarters and four-fifths of the errors made by the linear models involve the Neutral class, which confirms it as the central difficulty of the task.

1.7 Organisation of the Report

Chapter 2 gives the background and related work. Chapter 3 states the specification and design of the platform. Chapter 4 describes the methodology, from data collection to deployment. Chapter 5 presents and evaluates the results. Chapter 6 discusses gaps and future work, and Chapter 7 concludes. The references, appendices and a glossary follow.

Chapter 2: Background

2.1 Sentiment Analysis and Opinion Mining

Sentiment analysis, also called opinion mining, is a subfield of natural language processing concerned with determining the polarity of opinion expressed in text [1], [2]. Review sentiment analysis is one of its most common applied settings because review platforms provide a continuous supply of opinionated text, often paired with a numeric rating that can serve as a weak indicator of polarity. Approaches fall broadly into three families: lexicon-based methods, classical supervised machine learning, and neural or transformer-based models.

2.2 The Problem Addressed

Multi-outlet service businesses receive far more reviews than any team can read. The problem addressed here is to convert this stream into reliable sentiment labels and interpretable drivers at scale. Doing so well requires solving several sub-problems that earlier coursework-style studies often skip. First, star ratings are an imperfect proxy for sentiment: a three-star review can be clearly positive or clearly negative in its wording, and a five-star review can contain a complaint. Second, scraped review data contains duplicates and auto-generated tags that inflate performance if they are not removed. Third, the Neutral class is both a minority and semantically ambiguous, so a classifier can achieve high accuracy while failing on it entirely. This project addresses all three.

2.3 Related Work

2.3.1 Lexicon-based methods

Lexicon-based tools score text using a dictionary of words with pre-assigned polarity. VADER (Valence Aware Dictionary and sEntiment Reasoner) combines a sentiment lexicon with rules for negation, intensifiers and punctuation and needs no training data, which makes it a popular baseline for short informal text [3]. Its limitation is that it cannot adapt to domain vocabulary. In a fast-food setting, domain cues such as cold food or a long wait are strongly negative, and a general lexicon may under-weight them.

2.3.2 Classical supervised learning

Pang, Lee and Vaithyanathan showed that supervised classifiers over word features outperform hand-built lexicons for movie-review polarity [4]. TF-IDF weighting [5] combined with linear models remains a strong, cheap and interpretable baseline: Support Vector Machines [6], Logistic Regression and Naive Bayes [7] were each competitive, and Wang and Manning found that unigram and bigram features with linear models are hard to beat on many sentiment tasks [8]. These models train in seconds, expose per-term weights that explain their decisions, and run in well under a millisecond per review. Their weakness is that a bag-of-words representation ignores word order and context.

2.3.3 Transformer-based models

The transformer architecture, built on self-attention [11], underlies BERT [9], a bidirectional encoder pre-trained on large corpora and fine-tuned for downstream tasks. DistilBERT compresses BERT through knowledge distillation into a model with about 40% fewer parameters (roughly 66 million) that retains most of its language understanding while running faster [10]. Fine-tuned transformers capture negation, contrast and context, which is exactly what ambiguous, mixed-sentiment reviews demand, but they cost more to train and serve. The Hugging Face Transformers library provides the tooling used here [13].

2.3.4 Weak supervision and pseudo-labelling

Pseudo-labelling assigns model-predicted labels to unlabelled data so that a small, carefully annotated sample can be extended to a large corpus [15]. It reduces annotation cost but introduces a dependency between the labels and the model that produced them. Agreement statistics such as Fleiss' kappa [14] are used to establish how reliable the human anchor is, and the resulting limitations must be reported openly. Section 5.10 discusses them for this project.

2.3.5 Imbalanced classification

When one class is much rarer than the others, accuracy rewards ignoring it. He and Garcia survey remedies such as cost-sensitive learning and resampling, and stress the use of metrics that expose minority-class performance [16]. This project uses cost-sensitive class weights (in Logistic Regression, Linear SVM and the transformer loss) and evaluates with macro-averaged precision, recall and F1.

2.4 Gap and Contribution

Published review-sentiment studies typically use star-rating proxies as ground truth, report accuracy on a single split, and compare only one or two model families. This project contributes

(i) A documented hybrid labelling protocol with measured annotator agreement,

(ii) An explicit label-leakage audit, including the isolation of Google auto-generated tags,

(iii) A like-for-like comparison of classical, lexicon and transformer models under one leakage-free protocol with cross-validation, calibration-aware and per-class metrics, and

(iv) A deployed platform that makes the models usable by non-technical staff.

Chapter 3: Specification and Design

This chapter states what the system is required to do (the specification) and describes, at a high level, how the requirements are met (the design). Space is reserved for the diagrams that document the design; they will be inserted into the frames provided.

3.1 Users and Stakeholders

Two groups of users are considered. **Business users** (customer-experience analysts and outlet managers) want to see how customers feel, which topics drive dissatisfaction, and how sentiment changes across time or outlets, without writing code. **Technical users** (data scientists and maintainers) want to reproduce experiments, compare models and extend the pipeline. A third actor, the model service, is an internal component that loads the serialised models and returns predictions.

3.2 Functional Requirements

Table 1 lists the functional requirements. They are derived from the capabilities implemented in the platform's views and services.

ID	Requirement
FR1	Classify a single free-text review as Negative, Neutral or Positive and return class probabilities.
FR2	Let the user choose among the available models (classical and DistilBERT) and compare their outputs side by side in a multi-model sandbox.
FR3	Accept bulk review files in CSV, XLSX or JSON format, score every review and summarise the results.
FR4	Present executive KPIs and the overall sentiment share on an overview page.
FR5	Provide a filterable, paginated explorer of individual reviews with their labels.
FR6	Display the model benchmark table and confusion matrices.
FR7	Present deep-dive analytics and a gallery of diagnostic figures.
FR8	Generate and export an executive report.
FR9	Expose the most influential terms behind a prediction (explainability).
FR10	Re-run the complete evaluation pipeline end to end and regenerate all result files and figures.

Table 1 - Functional requirements of the platform

3.3 Non-Functional Requirements

Quality attribute	Requirement
Performance	Real-time inference: the Linear SVM classifies a review in under a millisecond, and DistilBERT in roughly 8.5 ms per review on a CPU.
Reproducibility	A single command reproduces the split, training, evaluation and all artefacts from the same fixed configuration.
Integrity	No information from the validation or test sets may influence vectoriser fitting, feature selection or hyper-parameter choice.
Interpretability	Linear models must expose term weights so that predictions can be explained to non-technical users.
Modularity	Data loading, features, models, evaluation, explainability and robustness are separate modules with a REST boundary between the model service and the interface.
Usability	A browser-based dashboard with no installation for end users, supporting single and bulk input.
Maintainability	Documented labelling decisions, versioned scripts and a dependency file (requirements.txt).

Table 2 - Non-functional requirements of the platform

3.4 Use Case Diagram

The main actors are the business user, the technical user and the model service. The principal use cases are: predict the sentiment of a review, compare models, upload a batch of reviews, browse and filter reviews, view analytics and KPIs, export a report, and run the evaluation pipeline. Figure 1 is reserved for this diagram.

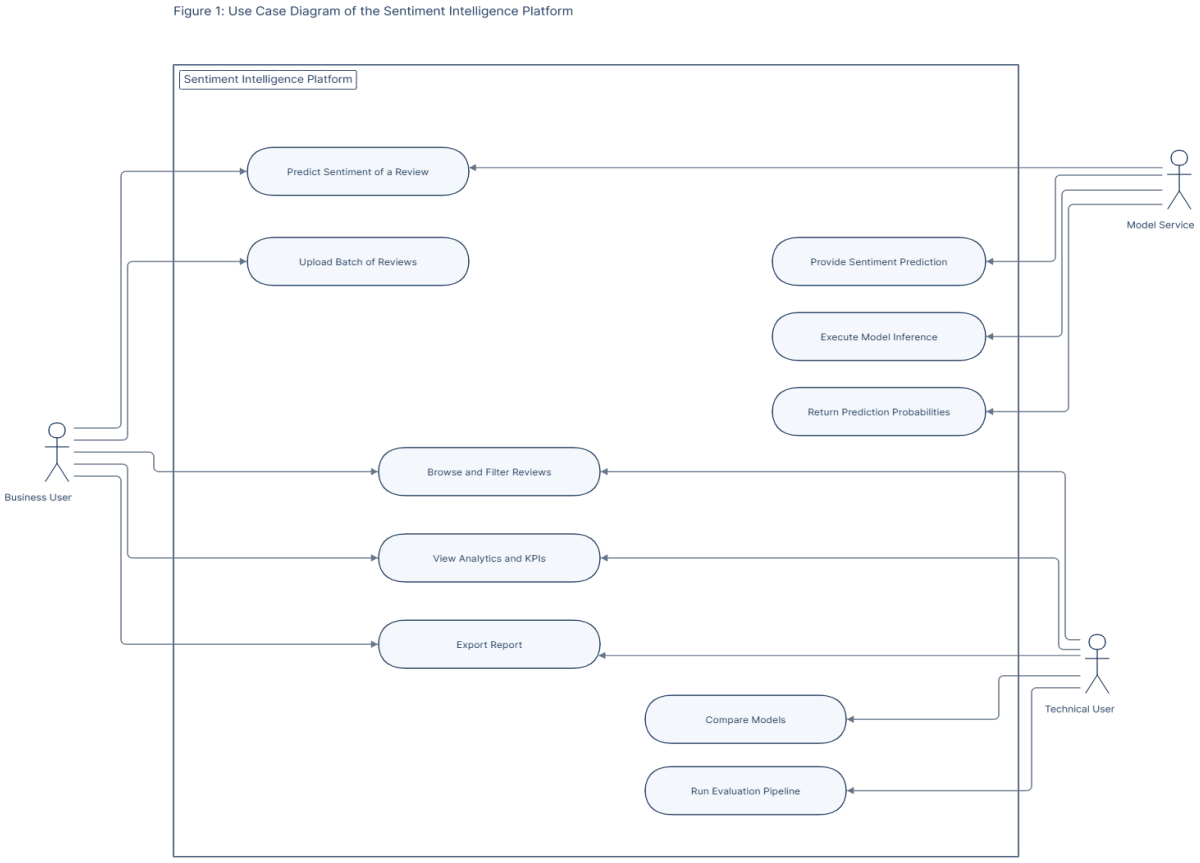


Figure 1 - Use case diagram of the Sentiment Intelligence Platform.

3.5 System Architecture

The platform is organised in four layers, shown schematically in Figure 2.

- **Data layer:** the raw, cleaned and labelled corpora (DATASET.csv, cleaned_reviews.csv and final_pseudo_labeled_dataset.csv) and the labelling workspace.
- **Machine-learning layer:** the modular Python library (src) for data loading, feature engineering, model training, evaluation, cross-validation, error analysis, efficiency profiling, ablation, robustness testing and explainability, together with the serialised models and the TF-IDF vectoriser.
- **Service layer:** a FastAPI REST server (server.py) that loads the models once at start-up, performs text preprocessing and vectorisation, and returns predictions and metrics as JSON.
- **Presentation layer:** a React 18 and Vite single-page application with views for Overview, Predictor, Batch Upload, Analytics, Reviews, Models and Reports, communicating with the server through an Axios client.

Figure 2: System Architecture of the Sentiment Intelligence Platform

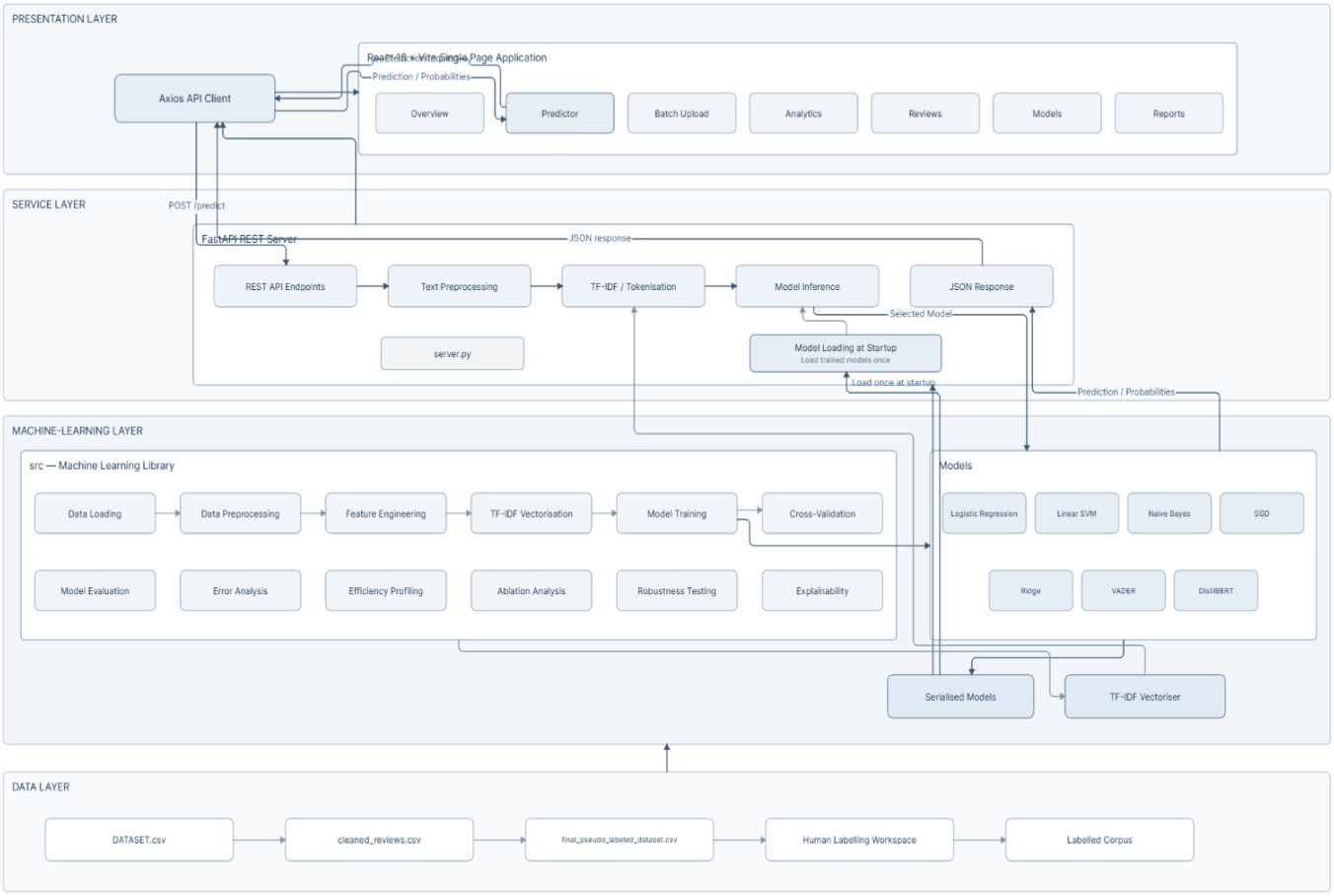


Figure 2 - System Architecture of Sentiment Intelligence Platform.

3.6 Sequence of a Prediction Request

A live prediction flows from the Predictor view through the Axios client to the FastAPI endpoint. The server cleans the text, converts it to TF-IDF features (or to token identifiers for DistilBERT), invokes the selected model, and returns the predicted class with its probabilities and influential terms, which the interface renders. Figure 3 is reserved for the corresponding sequence diagram.

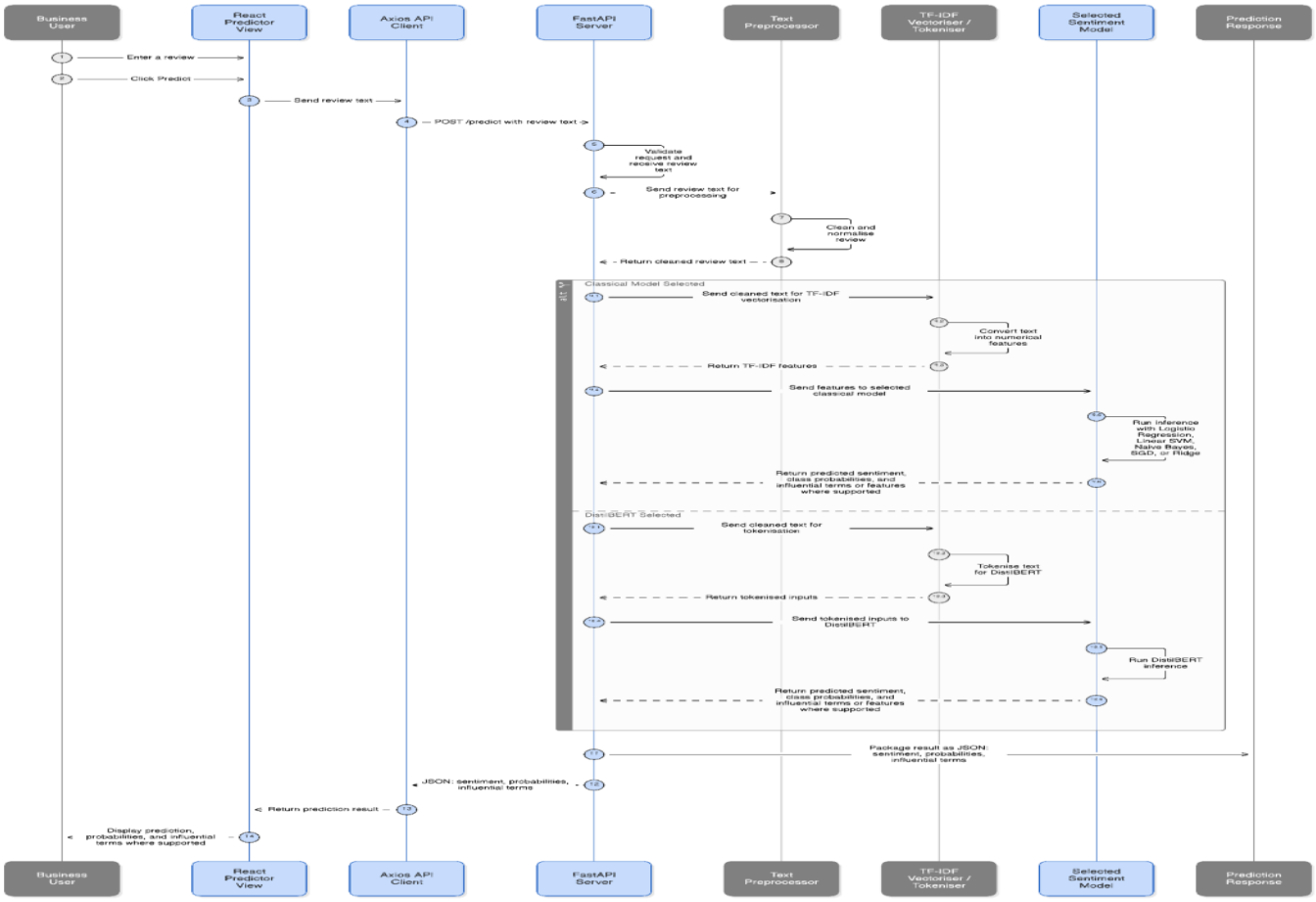


Figure 3 - Sequence diagram of the Sentiment Intelligence Platform.

Chapter 4: Methodology

This chapter describes the method at the level of implementation: how the data were collected, cleaned and labelled, how features were engineered, how models were selected, trained and tested, and how the results were integrated into a deployed platform. Figure 5 gives an overview of the complete pipeline.

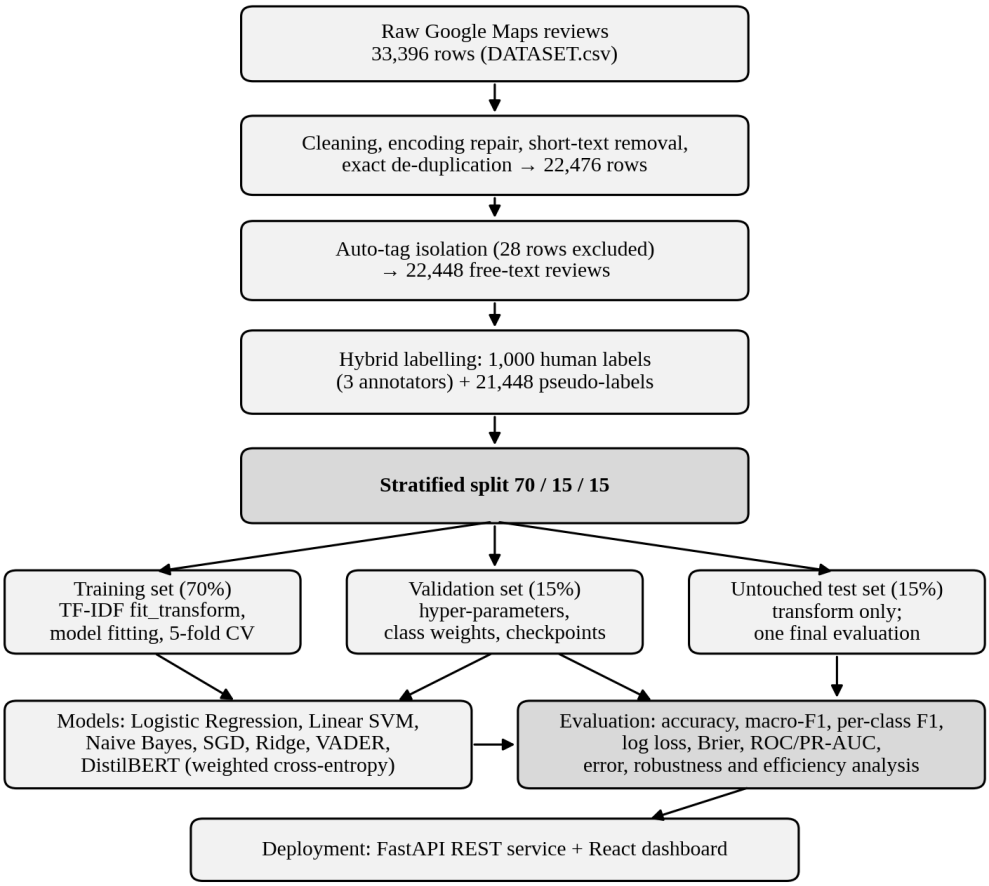


Figure 4 - Overview of the experimental and deployment pipeline

4.1 Data Collection

The dataset consists of 33,396 customer reviews of McDonald's outlets scraped from Google Maps [18] before the implementation phase of this project. The raw file, DATASET.csv, covers 40 distinct outlets in the United States, all categorised as "Fast food restaurant". Each raw record contains the reviewer identifier, store name, category and address, latitude and longitude, the outlet's aggregate rating count, the review's star rating stored as text (for example "5 stars"), a relative timestamp (for example "3 months ago") and the free-text review. The raw file was read with the cp1252 encoding to recover characters that were mis-encoded during scraping. Table 3 lists the fields of the cleaned corpus.

Field	Description
reviewer_id	Anonymised identifier of the reviewer
store_name, store_address, category	Outlet metadata
latitude, longitude	Geographic coordinates of the outlet
rating_num	Star rating given by the reviewer (1 to 5)
review, review_clean	Original and cleaned review text
days_ago, derived_date	Approximate age of the review parsed from the relative timestamp
is_autotag	Flag marking Google's auto-generated placeholder reviews
sentiment	Target label: Negative, Neutral or Positive
confidence, label_source	Label confidence and whether the label is human or model-generated
needs_human_review, split_role	Quality-control flag and the role of the row (for example, gold test)

Table 3 - Principal Fields of the cleaned and labelled review corpora

4.2 Data Cleaning and Preprocessing

The raw data passed through a ten-stage cleaning and labelling flow, summarised in Table 4. The cleaning notebook and the data-pipeline methodology report document every stage.

No.	Operation	Effect
1	Repair CP1252/UTF-8 encoding artefacts (mojibake) and normalise the store name	Corrupted characters removed
2	Parse the textual star rating into a numeric 1 to 5 value	rating_num
3	Convert the reviewer rating count to an integer	Numeric feature
4	Convert relative time ("3 months ago") to days_ago and derived_date	Temporal analysis enabled
5	Remove empty or very short reviews (fewer than 3 characters)	179 rows removed
6	Flag Google auto-generated rating tags (is_autotag)	5,167 rows flagged
7	Remove exact duplicate reviews	10,741 rows removed
8	Save the cleaned master dataset	22,476 rows
9	Run the 3-annotator pilot and the pseudo-labelling pipeline	Fleiss' kappa 0.6287
10	Save the final labelled corpus, excluding auto-tag rows	22,448 rows

Table 4 - The ten-stage data cleaning and preprocessing flow

The row counts reconcile exactly: 33,396 raw rows, less 10,741 duplicates and 179 unusable rows, gives the 22,476 rows of the cleaned master dataset. Removing the 28 auto-tag rows that survived de-duplication gives the 22,448 rows of the final corpus.

4.2.1 Auto-generated rating tags and label leakage

Inspection of the most frequent review texts showed that when a Google Maps user leaves a star rating without a comment, Google fills the review field with a generic tag that mirrors the rating, such as "excellent", "good", "average" or "poor". Such rows are not opinions: a model trained on them would learn the tag-to-label shortcut and report inflated performance. In total 5,167 rows were flagged. Nearly all were exact duplicates and were removed during de-duplication, and the 28 that remained are flagged with is_autotag and excluded from text modelling. Figure 6 confirms that every one of the 22,448 reviews retained for modelling is genuine free text. Two further leakage controls apply throughout: the TF-IDF vectoriser is fitted only on the training split, and metadata columns (star rating, address, timestamps) never enter the text feature matrix.

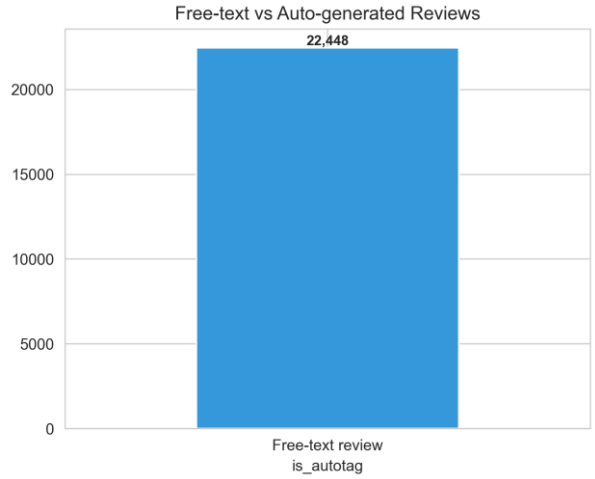


Figure 5 - Free-text versus auto-generated reviews after auto-tag isolation

.

4.3 Sentiment Labelling

An earlier version of this project derived labels from star ratings (one and two stars as Negative, three as Neutral, four and five as Positive). That rating-proxy dataset is archived as a superseded baseline and is not used for any result in this report. Star ratings are a coarse signal of what a reviewer wrote, so the final corpus uses a hybrid labelling protocol.

- **Human pilot.** A sample of 1,000 reviews was independently annotated by three annotators following written guidelines that define the boundary cases between classes. Their agreement was measured with Fleiss' kappa [14], giving 0.6287, which is substantial agreement on the Landis and Koch scale [17]. Krippendorff's alpha was also computed [Insert value from the labeling/reports folder]. The consensus labels form the human-labelled part of the corpus.
- **Pseudo-labelling.** The remaining 21,448 reviews were labelled by a model trained on the human-labelled reviews [Insert: the model used for pseudo-labelling, its confidence threshold and how disagreements were handled]. Each label carries a confidence score and a label_source, and low-confidence rows are flagged with needs_human_review.
- **Gold test reviews.** Human-labelled reviews reserved as gold_test are kept out of model fitting.

Table 5 gives the label composition and compares it with the class counts that the old star-rating mapping would have produced. The text-based labels place 1,513 fewer reviews in the Positive class and 1,172 more in the Negative class than the star mapping, a net shift that indicates many four- and five-star reviews contain critical wording. These are net class-level differences and do not identify individual relabelled reviews.

Class	Reviews	Share	Star mapping	Net difference
Negative	10,626	47.34%	9,454 (1 to 2 stars)	+1,172
Neutral	3,530	15.73%	3,189 (3 stars)	+341
Positive	8,292	36.94%	9,805 (4 to 5 stars)	−1,513
Total	22,448	100%	22,448	0

Table 5 - Final label distribution and comparison with the star-rating mapping.

Of the 22,448 labels, 1,000 (4.5%) are human labels and 21,448 (95.5%) are model pseudo-labels. This heavy reliance on pseudo-labels is the most important caveat of the study and is examined in Section 5.10.

4.4 Exploratory Data Analysis

Figure 7 shows the sentiment class distribution beside the star-rating distribution. The corpus is polarised: one-star (7,188) and five-star (6,176) reviews dominate, and the Neutral class (3,530 reviews) is clearly under-represented, so the target is imbalanced.

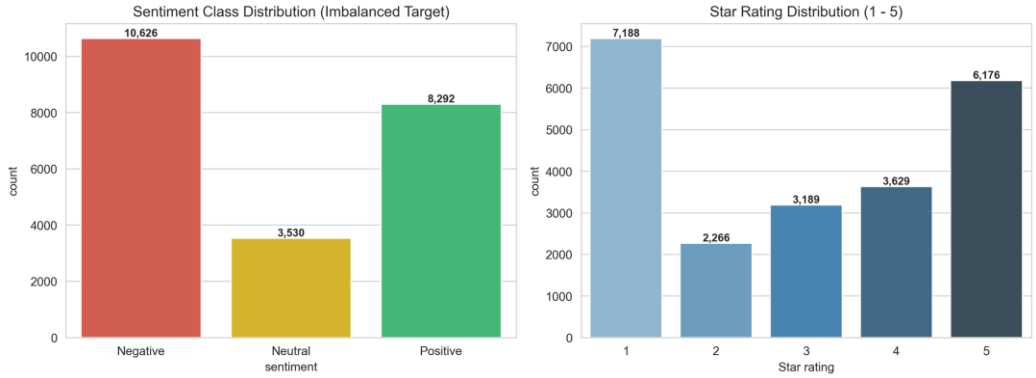


Figure 6 : Sentiment class distribution (left) and star-rating distribution (right)

Figure 8 compares review length across classes. Negative reviews are the longest (median about 118 characters), whereas Positive (about 53) and Neutral (about 42) reviews are much shorter. Dissatisfied customers therefore write more to describe what went wrong, and short reviews are more likely to be Neutral or Positive. The vertical axis is truncated at roughly 600 characters.

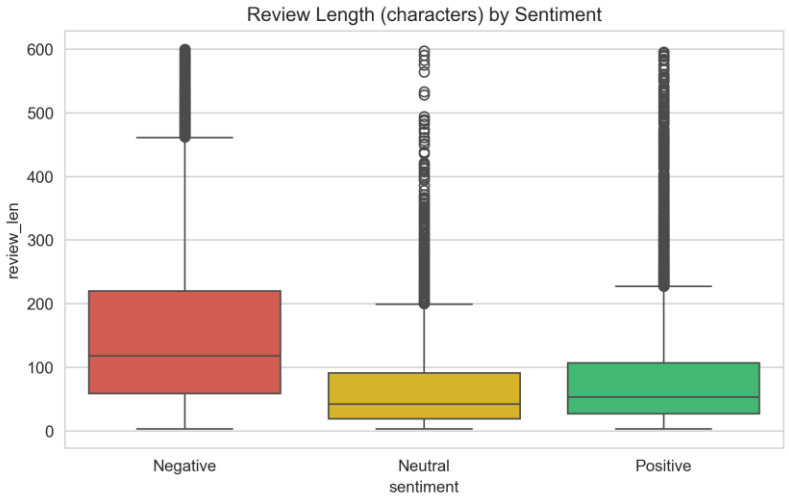


Figure 7 - Distribution of review length (characters) by sentiment class

Figure 9 shows sentiment composition by how long ago the review was posted. Negative reviews form the largest share in every period, so service complaints are a persistent pattern rather than a transient one. The 3 to 5 year bucket is the largest, with roughly 7,600 reviews.

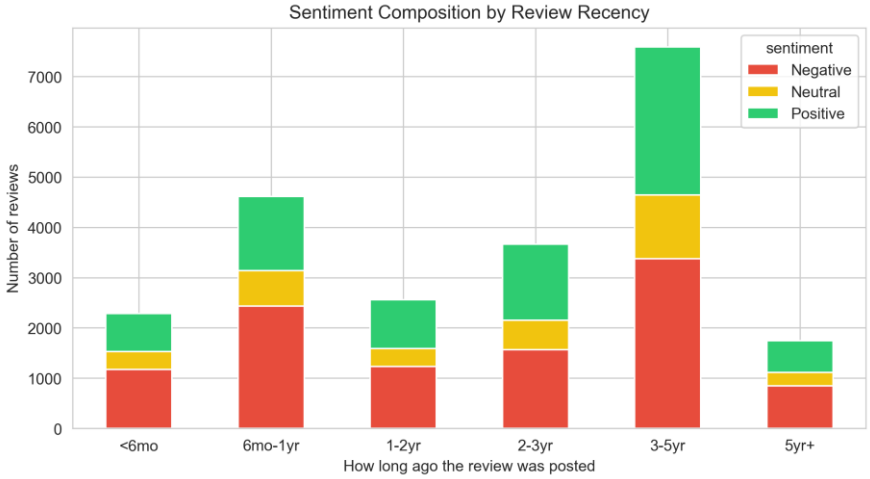


Figure 8 - Sentiment composition by review recency

Figure 10 shows word clouds for each class. Positive reviews are dominated by "good", "great", "clean", "friendly", "fast" and "staff". Negative reviews centre on "order", "drive thru", "employee", "rude", "time", "minute", "told" and "worst". Neutral reviews mix general terms ("food", "good", "one", "always", "busy") with hedged or mixed wording, which offers little distinctive vocabulary. The review counts in the panel titles (8,292, 3,452 and 10,622) are marginally lower than the full class sizes because the clouds are built from the preprocessed text.

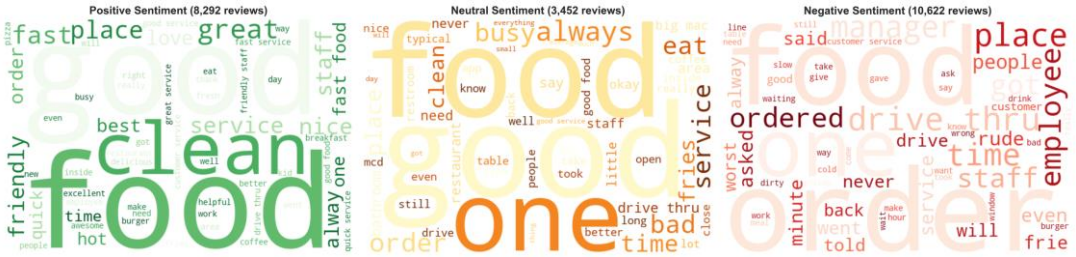


Figure 9 - Word clouds by sentiment class

Figure 11 lists the most frequent bigrams. Positive phrases are about speed and staff ("fast service", "friendly staff", "great service"). Negative phrases describe failures: "order wrong", "waited minutes", "food cold", "slow service", "long wait" and "horrible service". The Neutral class is dominated by product and place phrases ("big mac", "parking lot", "play area") and by hedging ("hit miss"). Note that "customer service" is the most frequent bigram in both the Positive and Negative classes, so raw bigram frequency alone does not discriminate sentiment, which is one reason supervised models are needed.

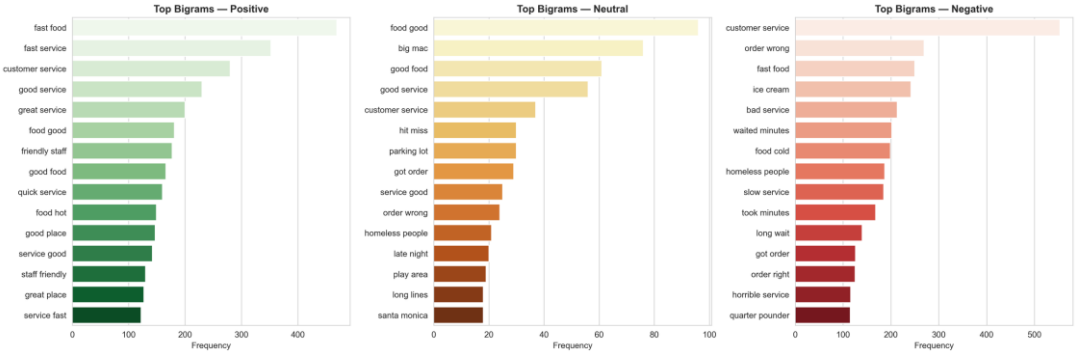


Figure 10 - Most frequent bigrams by sentiment class

4.5 Text Preprocessing and Feature Engineering

Reviews were lower-cased, stripped of URLs and non-alphabetic characters, and whitespace-normalised. English stop-words were removed together with a small set of domain terms that carry no sentiment (for example the brand name and the word "location"). Negation words such as "not", "no" and "never" were deliberately preserved, because they can reverse the polarity of a sentence and are highly informative.

For the classical models the cleaned text was converted to numeric features with Term Frequency-Inverse Document Frequency (TF-IDF) weighting [5], configured as in Table 6. Including bigrams allows the linear models to capture short polarity-bearing phrases such as "not good" or "fast service", and sublinear term-frequency scaling damps the effect of words that are repeated many times within one review. Feature selection is implicit: the vocabulary is capped at the 20,000 highest-frequency n-grams of the training split, and the ablation module (Section 5.9) examines the contribution of feature groups. DistilBERT does not use TF-IDF; it uses its own sub-word tokenizer and learns contextual representations.

Parameter	Value
N-gram range	Unigrams and bigrams
Maximum features	20,000
Term-frequency scaling	Sublinear (1 + log tf)
Fitting rule	fit_transform on the training split only; transform on validation and test
Excluded inputs	Star rating, store address, timestamps and other metadata

Table 6 -TF-IDF feature configuration

Simplified excerpt of the leakage-free feature and model code

```
vectorizer = TfidfVectorizer(ngram_range=(1, 2), sublinear_tf=True,
                             max_features=20000)
X_train = vectorizer.fit_transform(train_text)   # vocabulary from
training data only
X_val   = vectorizer.transform(val_text)
X_test  = vectorizer.transform(test_text)
svm = LinearSVC(class_weight="balanced").fit(X_train, y_train)
```

4.6 Data Splitting and Cross-Validation

The labelled corpus was divided by a three-way stratified split so that each subset preserves the class proportions: 70% for training, 15% for validation and 15% for the final test. The training set is used exclusively for model fitting and vocabulary construction. The validation set is used for hyper-parameter tuning, loss weighting and transformer checkpoint selection. The test set (3,319 reviews, as reflected in the confusion matrices of Chapter 5) is held out until final evaluation and is never exposed to feature selection, vectoriser fitting or tuning. In addition, classical models were assessed with five-fold stratified cross-validation, reported as mean and standard deviation of macro-F1, to show that their ranking does not depend on one lucky split.

4.7 Model Selection, Training and Testing

Seven approaches were compared, from a rule-based baseline to a fine-tuned transformer, so that the added value of each step in complexity could be measured. Table 7 summarises their configuration.

Model	Representation	Key configuration
Logistic Regression	TF-IDF	L2 regularisation, balanced class weights, probabilistic output
Linear SVM (LinearSVC)	TF-IDF	Balanced class weights
Multinomial Naive Bayes	TF-IDF	Laplace smoothing (alpha = 1.0), no cost-sensitive weighting
SGDClassifier	TF-IDF	Linear model trained by stochastic gradient descent
RidgeClassifier	TF-IDF	Linear model with L2-penalised least squares
VADER	Raw text lexicon	Standard polarity thresholds on the compound score; no training
DistilBERT	Sub-word tokens	distilbert-base-uncased (about 66 million parameters), fine-tuned with class-weighted cross-entropy

Table 7 - Model suite and configuration

DistilBERT was fine-tuned on the training split with a class-weighted cross-entropy loss, so that errors on the Neutral minority are penalised more heavily, and the checkpoint with the highest validation macro-F1 was kept. Table 8 records the fine-tuning settings.

Setting	Value
Pre-trained model	distilbert-base-uncased
Loss	Class-weighted cross-entropy
Checkpoint selection	Highest validation macro-F1 (early stopping)
Epochs	3
Batch size	64
Maximum sequence length	128

Table 8 - DistilBERT fine-tuning settings

Simplified excerpt of the weighted transformer loss

```
loss_fn = torch.nn.CrossEntropyLoss(weight=class_weights)
loss = loss_fn(logits, labels)   # Neutral errors cost more than
majority-class errors
```

4.8 Evaluation Metrics

Because the Neutral class is only 15.73% of the corpus, accuracy can be high for a classifier that ignores it. A classifier that predicts only Negative and Positive can still reach roughly 84% accuracy with a Neutral F1 near zero. Macro-F1 is therefore the primary model-selection metric, supported by the measures in Table 9.

Metric	Definition and purpose
Accuracy	Share of test reviews classified correctly; dominated by the majority classes.
Macro precision, recall, F1	Unweighted mean of the per-class values; the primary criterion because each class counts equally.
Weighted F1	Mean of per-class F1 weighted by class size; reflects overall performance.
Per-class F1	F1 of Negative, Neutral and Positive separately; exposes minority-class failure.
Log loss	Penalises confident wrong probability estimates; lower is better.
Brier score	Mean squared error of the predicted probabilities; lower is better.
ROC-AUC, PR-AUC	Threshold-independent ranking quality (macro-averaged); higher is better.
Cross-validated macro-F1	Mean and standard deviation over five stratified folds; measures stability.

Table 9 - Evaluation metrics

4.9 Software and Hardware Requirements

Component	Specification
Language and data	Python, pandas, NumPy
Classical machine learning	scikit-learn [12]
Transformer	PyTorch and Hugging Face Transformers [13]; distilbert-base-uncased
Lexicon baseline	VADER [3]
Visualisation	Matplotlib, Seaborn, word-cloud generation
Backend	FastAPI [19] REST server (port 8000)
Frontend	React 18 [20], Vite, Axios (port 5173)
Library versions	[Insert from requirements.txt]
Training hardware	[Insert CPU, GPU, RAM]
Inference hardware for latency tests	CPU (about 8.5 ms per review for DistilBERT; sub-millisecond for Linear SVM)

Table 10 - Software and hardware environment

4.10 Model Integration and Deployment

The serialised models (best_sentiment_model.pkl, linear_svm_model.pkl, logistic_regression_model.pkl, naive_bayes_model.pkl), the fitted vectoriser (tfidf_vectorizer.pkl) and the fine-tuned DistilBERT directory are loaded once by the FastAPI server at start-up [Insert: which model best_sentiment_model.pkl contains]. The server exposes endpoints for single and batch prediction, model metrics and figures. The React application consumes them through an Axios client and provides seven views: Overview (executive KPIs and sentiment share), Predictor (live single prediction and a multi-model sandbox), Batch Upload (CSV, XLSX and JSON ingestion), Analytics (deep-dive charts and figure gallery), Reviews (filterable, paginated explorer), Models (benchmark table and confusion matrices) and Reports (executive report generator and exporter).

The complete evaluation is executable with one command (python run_evaluation_pipeline.py), which regenerates the split, trains and benchmarks all classical models, evaluates DistilBERT and VADER, runs cross-validation, error analysis and robustness tests, and exports all result tables and figures. The repository layout is given in Appendix A.

4.11 Difficulties Encountered and Solutions

Difficulty	Solution
Encoding artefacts (mojibake) corrupted store names and review text.	Read the file with cp1252, sanitised replacement characters and merged corrupted store-name variants.
Google auto-generated tags mirrored the star rating and would leak the label.	Flagged them (is_autotag), removed duplicates and excluded the rest from text modelling.
Heavy duplication (10,741 exact duplicates) inflated apparent data size.	Exact de-duplication before splitting, so that duplicates cannot span train and test.
Star ratings were an unreliable proxy for sentiment.	Replaced by the hybrid human and pseudo-labelling protocol with measured agreement.
The Neutral class is a small, ambiguous minority.	Class weights, macro-F1 as the selection metric and a transformer with weighted loss.
DistilBERT is far slower than linear models at inference.	Kept linear models in the platform as low-latency options and profiled latency explicitly.
Risk of information leaking into the test set.	Three-way split; vectoriser fitted on training data only; single final test evaluation.

Table 11 - Difficulties encountered during implementation and how they were resolved

Chapter 5: Results and Evaluation

This chapter shows how the system was demonstrated to work as intended. All classifier results are measured on the untouched test set unless stated otherwise, and the metrics are those defined in Section 4.8.

5.1 Overall Performance

Table 12 and Figure 12 report the headline metrics. The fine-tuned DistilBERT model is the strongest approach on every measure, with an accuracy of 0.9093 and a macro-F1 of 0.8906. Logistic Regression and Linear SVM form a second tier with macro-F1 values of 0.8233 and 0.8195. SGDClassifier (0.8111) and RidgeClassifier (0.8008) follow closely, Multinomial Naive Bayes falls to 0.5863, and the VADER baseline reaches 0.5539.

Model	Accuracy	Macro precision	Macro recall	Macro F1	Weighted F1
Logistic Regression	0.8569	0.8154	0.8403	0.8233	0.8618
Linear SVM	0.8602	0.8187	0.8205	0.8195	0.8607
Multinomial Naive Bayes	0.7749	0.8552	0.6159	0.5863	0.7199
SGDClassifier	0.8530	0.8143	0.8088	0.8111	0.8528
RidgeClassifier	0.8382	0.7939	0.8143	0.8008	0.8427
VADER (baseline)	0.6171	0.5665	0.5668	0.5539	0.6207
DistilBERT (transformer)	**0.9093**	**0.8793**	**0.9064**	**0.8906**	**0.9110**

Table 12 - Test-set performance of all models

Note that Multinomial Naive Bayes has a higher macro precision (0.8552) than every other classical model but the lowest macro recall (0.6159) among the supervised models. It is precise when it does predict a class, but it fails to recognise many Neutral reviews. Its accuracy of 0.7749 therefore overstates its usefulness: the gap between accuracy and macro-F1 is 0.1886, compared with 0.0336 for Logistic Regression and 0.0187 for DistilBERT.

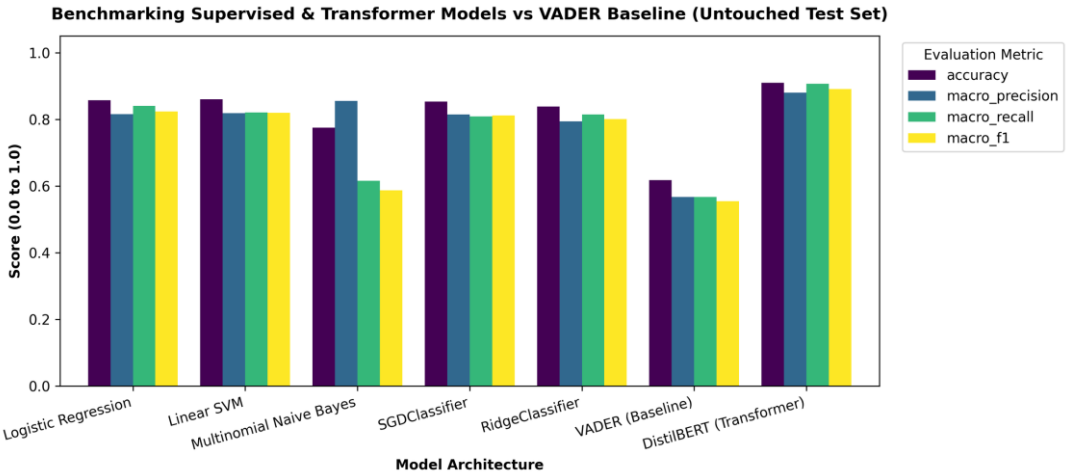


Figure 11 - Accuracy, macro precision, macro recall and macro-F1 of all models on the untouched test set.

5.2 Per-Class Performance

Table 13 breaks F1 down by class. All models handle the Negative and Positive classes well (F1 between 0.83 and 0.93 for every model except VADER), and the models differ mainly on the Neutral class. DistilBERT raises Neutral F1 to 0.8139, compared with 0.6819 for Logistic Regression, 0.6686 for Linear SVM, 0.0894 for Naive Bayes and 0.2963 for VADER. Improving the Neutral class is therefore where the transformer earns most of its advantage: the macro-F1 gain of 0.0673 over Logistic Regression is driven mainly by the 0.1320 gain on Neutral.

Model	Negative F1	Neutral F1	Positive F1
Logistic Regression	0.8985	0.6819	0.8896
Linear SVM	0.9012	0.6686	0.8887
Multinomial Naive Bayes	0.8373	0.0894	0.8323
SGDClassifier	0.8931	0.6576	0.8826
RidgeClassifier	0.8877	0.6494	0.8654
VADER (baseline)	0.6627	0.2963	0.7026
DistilBERT (transformer)	**0.9275**	**0.8139**	**0.9303**

Table 13 - F1-score per class on the test set

The multinomial Naive Bayes result illustrates the danger of accuracy. As configured (Laplace smoothing, no cost-sensitive weighting) it favours the frequent classes and almost never predicts Neutral. The two class-weighted linear models avoid this failure, which supports the decision to use balanced class weights for the minority class.

5.3 Stability under Cross-Validation

Table 14 reports five-fold cross-validated macro-F1 for the classical models. Logistic Regression and Linear SVM have the same mean (0.8294) and similar standard deviations (0.0065 and 0.0057), so the

small differences between them on the single test split (0.8233 versus 0.8195) are within cross-validation variability and should not be interpreted as a real ranking. SGDClassifier (0.8178) and RidgeClassifier (0.8115) are slightly lower, and Naive Bayes is clearly lower (0.5871). The standard deviations are all below 0.01, so the results are stable across folds. Cross-validation was not applied to VADER, which has no trainable parameters, or to DistilBERT, which was evaluated on the single validation-selected hold-out split.

Model	Macro F1 (mean ± SD)
Logistic Regression	0.8294 ± 0.0065
Linear SVM	0.8294 ± 0.0057
SGDClassifier	0.8178 ± 0.0086
RidgeClassifier	0.8115 ± 0.0058
Multinomial Naive Bayes	0.5871 ± 0.0082

Table 14 - Five-fold cross-validated macro-F1 (mean ± standard deviation)

5.4 Probabilistic Quality

Table 15 compares the models on probability-based measures. DistilBERT again leads: it has the lowest log loss (0.2642) and Brier score (0.0469) and the highest ROC-AUC (0.9807) and PR-AUC (0.9481). Among the classical models, Logistic Regression has the best log loss (0.4080), Brier score (0.0740) and ROC-AUC (0.9632), so it is the better choice when calibrated probabilities matter. VADER's log loss of 8.31 is very high, which is consistent with a rule-based tool that emits hard labels rather than calibrated probabilities, so confident mistakes are penalised heavily; its values are not comparable in kind with those of the probabilistic models.

Model	Log loss	Brier	ROC-AUC	PR-AUC
Logistic Regression	0.4080	0.0740	0.9632	0.8960
Linear SVM	0.4281	0.0764	0.9617	0.8910
Multinomial Naive Bayes	0.5626	0.1033	0.9314	0.8320
SGDClassifier	0.4698	0.0835	0.9581	0.8870
RidgeClassifier	0.5807	0.1038	0.9530	0.8757
VADER (baseline)	8.3098	0.3597	0.7055	0.5284
DistilBERT (transformer)	**0.2642**	**0.0469**	**0.9807**	**0.9481**

Table 15 - Probability-based metrics on the test set

5.5 Confusion Matrices and Error Analysis

Figure 14 shows the confusion matrices of Logistic Regression and Linear SVM on the 3,319 test reviews, and Table 16 gives the resulting per-class precision, recall and F1. Both linear models classify Negative and Positive reviews with precision and recall between 0.85 and 0.93. The Neutral class is the

weak point: Logistic Regression achieves a Neutral recall of 0.784 but a precision of only 0.604, because 264 Negative and Positive reviews are wrongly predicted as Neutral (140 and 124 respectively). Linear SVM is more conservative, with a Neutral precision of 0.659 and a recall of 0.678.

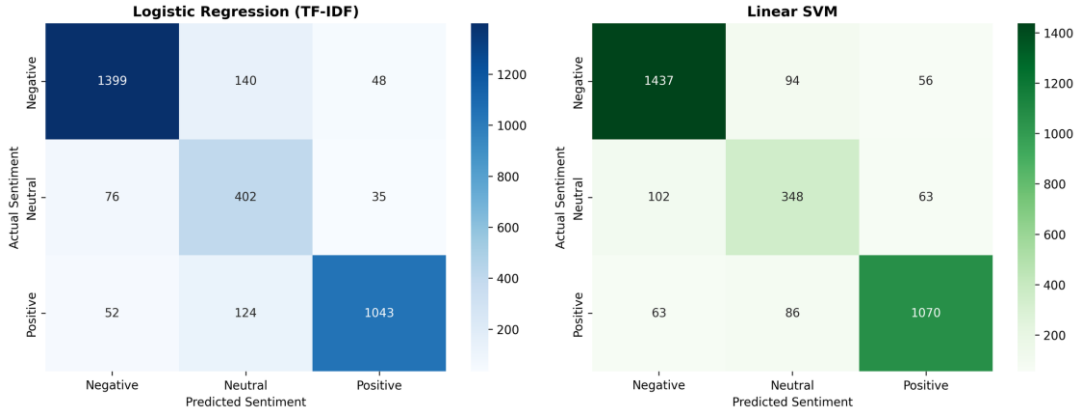


Figure 12 - Confusion matrices of Logistic Regression (left) and Linear SVM (right) on the test set

Model	Class	Precision	Recall	F1	Support
Logistic Regression	Negative	0.916	0.882	0.899	1,587
	Neutral	0.604	0.784	0.682	513
	Positive	0.926	0.856	0.890	1,219
Linear SVM	Negative	0.897	0.905	0.901	1,587
	Neutral	0.659	0.678	0.669	513
	Positive	0.900	0.878	0.889	1,219

Table 16 - Per-class precision, recall and F1 for the linear models, computed from the confusion matrices.

Of the 475 errors made by Logistic Regression, 375 (79%) involve the Neutral class, either as the true label (111 errors) or as the wrong prediction (264 errors); only 100 errors are direct Negative-Positive confusions. For Linear SVM the corresponding share is 345 of 464 errors (74%). Sentiment classification here is thus largely a problem of deciding where mixed or lukewarm opinion begins and ends, not of separating good from bad. Typical failures are contrastive sentences such as "the food was good but the staff were rude", and sarcasm such as "great, another two-hour wait", which bag-of-words models cannot resolve and which the contextual DistilBERT model handles better.

5.6 Most Predictive Terms

Figure 14 shows the largest coefficient weights of the linear model for each class, which gives a direct explanation of what drives predictions. Negative predictions are driven by service and quality failures: "worst", "slow", "rude", "dirty", "horrible", "cold", "terrible", "poor" and "minutes". Positive predictions are driven by "great", "friendly", "love", "fast", "quick", "nice", "hot", "clean", "awesome" and "best", reflecting staff behaviour, speed and cleanliness. The Neutral class has a weaker and less semantic

signature: its strongest weights ("kinda", "have not", "hit", "fine") are hedging expressions, and several others ("pies", "mickey", "condiments", "options") are topical rather than evaluative. The largest Neutral weight (about 2.8) is much smaller than the largest Negative (about 4.1) and Positive (about 7.5) weights, which explains why linear models struggle with it. The term "donalds" appears among the Negative predictors, a residue of the brand name that suggests the stop-word list should also remove split brand tokens.

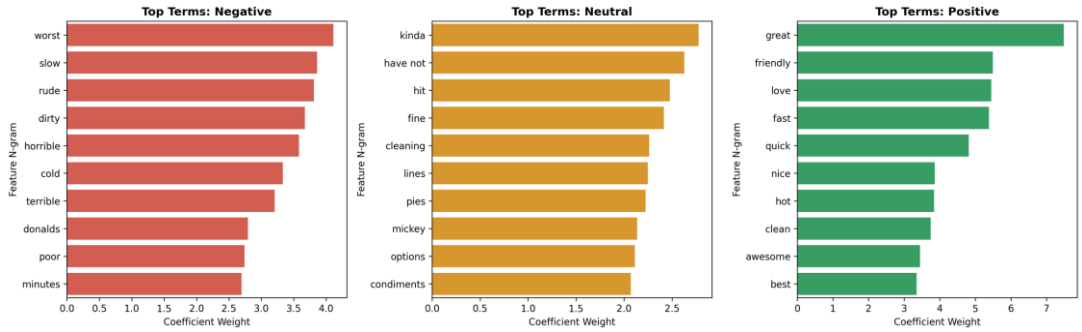


Figure 13 : Top predictive terms per sentiment class from the linear model coefficients

5.7 Lexicon Baseline (VADER)

Figure 15 plots VADER's compound score against the star rating. The median score rises steadily from about −0.46 for one-star reviews to about 0.63 for five-star reviews, so the review text does carry a sentiment signal that agrees with the ratings. The boxes overlap heavily, though: many one- and two-star reviews score above zero, and some five-star reviews score below it. On the test set VADER reaches an accuracy of 0.6171 and a Neutral F1 of only 0.2963, roughly 29 points of accuracy below DistilBERT and 24 below Logistic Regression. This quantifies the value of domain-specific training over a general-purpose lexicon.

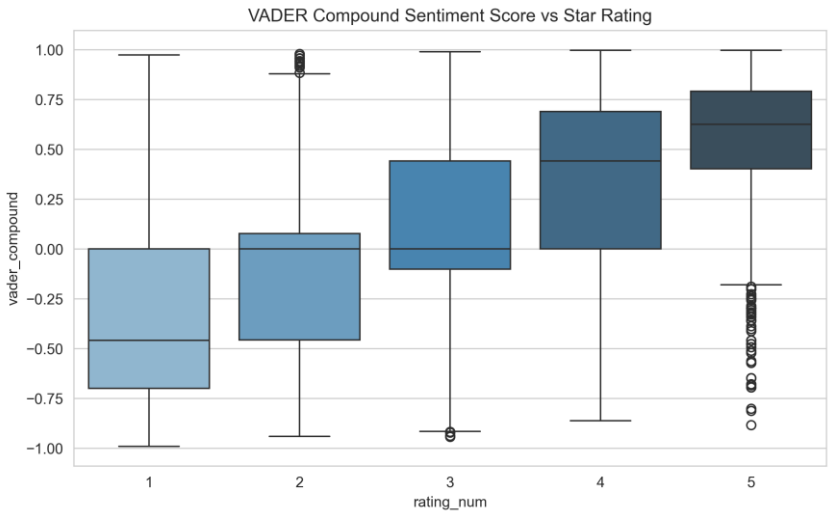


Figure 14 - VADER compound sentiment score by star rating

Figure 16 shows VADER's confusion matrix over an 8,000-review evaluation sample. Its dominant error is to label Negative reviews as Neutral (777) or Positive (890), which shows how often a lexicon misses

sarcasm, negation and domain-specific complaints and affects 43.6% of Negative reviews, and it labels 542 of the 1,209 Neutral reviews (44.8%) as Positive. [Verify before submission: this matrix is computed over 8,000 reviews and gives an accuracy of 0.6305, whereas the VADER row in Table 12 (0.6171) comes from the results file. Regenerate the figure on the same evaluation set as the table, or state the sample explicitly.]

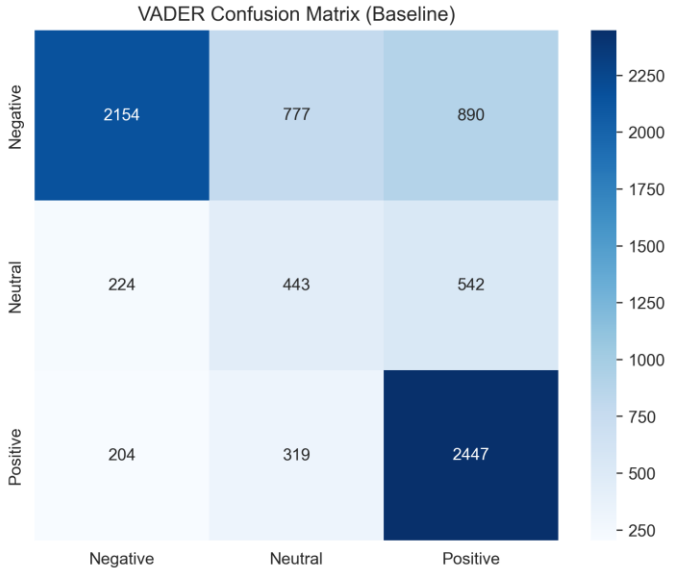


Figure 15 - Confusion matrix of the VADER baseline (rows: actual class; columns: predicted class).

5.8 Efficiency

The accuracy advantage of DistilBERT comes at a cost in speed. On a CPU, Linear SVM classifies a review in under a millisecond, whereas DistilBERT needs roughly 8.5 milliseconds per review, so the transformer is about an order of magnitude slower. For interactive use and for batch scoring of tens of thousands of reviews both are acceptable, but for very high throughput or for constrained edge devices the linear models are the practical choice. The platform therefore exposes both families and lets the user choose. Detailed latency, throughput and memory profiles are in results/efficiency_benchmark.csv.

5.9 Robustness, Ablation and Error Diagnostics

Three further analyses are part of the evaluation pipeline: a robustness suite that injects typos and noise into the test text, a feature and loss ablation study, and an error-analysis module that exports every misclassified review for inspection. Their outputs are stored in results/robustness_results.csv, results/ablation_results.csv and results/error_analysis.csv and are discussed in the Final Audit Report.

Section	Analysis / Finding	Result / Evidence
Robustness – Typo & Noise	Models were evaluated on 21 challenge examples covering negation, intensifiers, mixed sentiment, sarcasm, short/long text, spelling noise, and punctuation.	Logistic Regression: **0.6310**; Linear SVM: **0.7774**; Multinomial NB: **0.6556**; SGDClassifier: **0.7242**; RidgeClassifier: **0.6759**; VADER: **0.5040**; DistilBERT: **0.6794** macro-F1.
Robustness – Key finding	Performance varies substantially across linguistic challenges.	Linear SVM achieved **0.7774** macro-F1 on the supplied robustness challenge set, while VADER achieved **0.5040**. These are robustness-set scores; the **change from clean-test macro-F1 cannot be calculated** from the supplied data because the clean-test values are not provided.
Ablation – Class weights	Removing class weights was compared using unigram and bigram Linear SVM configurations.	Unigram: **0.8330 → 0.8369** with class weights (**+0.0039**). Bigram: **0.8225 → 0.8300** with class weights (**+0.0075**).
Ablation – Bigrams	Replacing unigrams with bigrams was evaluated under both weighting conditions.	Unweighted: **0.8330 → 0.8225** (**−0.0105**). Balanced: **0.8369 → 0.8300** (**−0.0069**).
Ablation – Trigrams & sublinear TF	Configuration E combines trigrams, sublinear TF, and balanced weights.	**0.8312** macro-F1, compared with **0.8300** for configuration D (**+0.0012**). The independent effect of sublinear TF cannot be isolated from the supplied configurations.
Error diagnostics	The dominant errors involve ambiguous polarity, contrastive clauses, and negation.	Neutral Ambiguity / Borderline Polarity: 114; Mixed Sentiment / Contrastive Clause: 56; Negation Interaction: 55; Domain Nuance / Uncategorized: 41; Very Long / Narrative Review: 18; Very Short Text: 17.

Table 17 - Summary of robustness and ablation results

5.10 Limitations and Threats to Validity

- **Pseudo-label dependence.** 95.5% of the labels are model-generated. Models are scored against these labels, so measured accuracy reflects agreement with the labelling procedure as well as with true sentiment. If a model in the benchmark resembles the labeller, its score may be optimistic. A human-only evaluation would give the least biased estimate.
- **Test-set composition.** The results are computed on a random 15% split of the labelled corpus (3,319 reviews). The number of human-labelled gold reviews in that split determines how much of the test score is measured against human truth.
- **Neutral-class ambiguity.** Annotator agreement (kappa 0.6287) is substantial rather than near-perfect, and most disagreement is expected at the Neutral boundary. Some Neutral "errors" are therefore label noise rather than model failure.
- **Single split for DistilBERT and VADER.** Only the classical models have cross-validation estimates, so the gap between DistilBERT and the linear models is measured on one split.
- **Domain and language.** The corpus is English-language, US-based fast-food reviews. The vocabulary is domain-specific and may not transfer to other sectors without retraining.
- **Approximate recency.** The days_ago values are estimates derived from relative time text.
- **Brand-token residue.** Split brand tokens such as "donalds" survive preprocessing and appear among predictive terms.

5.11 Discussion

Taken together, the results support four conclusions. First, sentiment in McDonald's reviews is learnable to a high standard: even a linear model on TF-IDF features exceeds 0.85 accuracy. Second, model choice matters most on the Neutral class, and a fine-tuned transformer is the only approach that brings Neutral F1 above 0.8. Third, accuracy is unreliable here: Naive Bayes looks acceptable by accuracy yet fails on the minority class. Fourth, the operational content is consistent across all analyses: dissatisfaction is driven by slow service, wrong orders, rudeness, cold food and cleanliness, while satisfaction is driven by friendly staff, speed, hot food and clean premises. These themes give managers concrete levers, and the platform makes them observable continuously.

Chapter 6: Future Work

6.1 Gaps in the Present Work

The principal gaps follow from the limitations in Section 5.10: the labels are mostly model-generated, cross-validation is missing for the transformer, and sentiment is assigned at review level, so a review that praises the food and criticises the service receives a single label.

6.2 Proposals for Enhancement

- **Larger human gold standard.** Annotate a stratified sample of Neutral and low-confidence reviews (those flagged needs_human_review) and report all models on this human-only test set, then use it to measure label noise directly.
- **Active learning.** Use model uncertainty to choose which reviews to annotate next, improving labels where they matter most.
- **Aspect-based sentiment.** Extract aspects such as speed, order accuracy, staff and cleanliness and score each separately, so that mixed reviews yield actionable, aspect-level signals.
- **Stronger and repeated transformer experiments.** Repeat DistilBERT fine-tuning over multiple seeds or folds, and compare larger models such as BERT or RoBERTa [9], [10].
- **Efficiency.** Apply quantisation or ONNX export to reduce DistilBERT latency, and consider a two-stage cascade in which the linear model handles confident cases and the transformer resolves uncertain ones.
- **Probability calibration and explainability.** Calibrate probabilities (temperature scaling or isotonic regression) and add token-level attribution for the transformer.
- **Geographic and temporal analytics.** Use the latitude and longitude of the 40 outlets to map sentiment by outlet, and monitor drift and complaint spikes over time with alerts.
- **Cross-domain generalisation.** Test the models on other review domains and languages to measure transfer.
- **Production hardening.** Add authentication, scheduled batch scoring, model versioning and monitoring to the platform.

Chapter 7: Conclusion

This project set out to build and rigorously compare approaches to three-class sentiment analysis of McDonald's Google Maps reviews and to deploy the outcome as a usable platform. A real-world corpus of 33,396 scraped reviews was audited and cleaned. A data-quality hazard, Google's auto-generated rating tags, was identified and removed before it could leak labels, and a hybrid labelling protocol with a measured annotator agreement of 0.6287 produced 22,448 labelled reviews.

Under a leakage-free 70/15/15 protocol, the fine-tuned DistilBERT model achieved the best results (accuracy 0.9093, macro-F1 0.8906, Neutral F1 0.8139). Logistic Regression and Linear SVM performed almost identically under cross-validation (macro-F1 0.8294) and offer sub-millisecond, interpretable inference, whereas Multinomial Naive Bayes and the VADER baseline confirmed that accuracy alone and generic lexicons are unreliable for this task. The Neutral class accounted for the large majority of the linear models' errors and remains the central challenge. The linguistic analysis showed that speed, order accuracy, staff behaviour and cleanliness drive customer sentiment, providing operational insight.

The validity of these findings is bounded by their reliance on pseudo-labels and by single-split evaluation of the transformer; both are documented and addressed by the proposals in Chapter 6. The deliverables, namely the cleaned and labelled datasets, the reproducible pipeline, the trained models, the audit reports and the FastAPI and React platform, provide a solid and extensible foundation for continuous customer-feedback monitoring in multi-outlet service businesses.

References

[1] B. Pang and L. Lee, "Opinion mining and sentiment analysis," Found. Trends Inf. Retr., vol. 2, no. 1–2, pp. 1–135, 2008.

[2] B. Liu, Sentiment Analysis and Opinion Mining. San Rafael, CA, USA: Morgan & Claypool, 2012.

[3] C. J. Hutto and E. Gilbert, "VADER: A parsimonious rule-based model for sentiment analysis of social media text," in Proc. 8th Int. AAAI Conf. Weblogs Social Media (ICWSM), Ann Arbor, MI, USA, 2014, pp. 216–225.

[4] B. Pang, L. Lee, and S. Vaithyanathan, "Thumbs up? Sentiment classification using machine learning techniques," in Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP), Philadelphia, PA, USA, 2002, pp. 79–86.

[5] G. Salton and C. Buckley, "Term-weighting approaches in automatic text retrieval," Inf. Process. Manage., vol. 24, no. 5, pp. 513–523, 1988.

[6] C. Cortes and V. Vapnik, "Support-vector networks," Mach. Learn., vol. 20, no. 3, pp. 273–297, 1995.

[7] A. McCallum and K. Nigam, "A comparison of event models for Naive Bayes text classification," in Proc. AAAI-98 Workshop Learn. Text Categorization, 1998, pp. 41–48.

[8] S. Wang and C. D. Manning, "Baselines and bigrams: Simple, good sentiment and topic classification," in Proc. 50th Annu. Meeting Assoc. Comput. Linguistics (ACL), Jeju Island, South Korea, 2012, pp. 90–94.

[9] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in Proc. NAACL-HLT, Minneapolis, MN, USA, 2019, pp. 4171–4186.

[10] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, "DistilBERT, a distilled version of BERT: Smaller, faster, cheaper and lighter," arXiv:1910.01108, 2019.

[11] A. Vaswani et al., "Attention is all you need," in Adv. Neural Inf. Process. Syst. (NeurIPS), vol. 30, 2017, pp. 5998–6008.

[12] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," J. Mach. Learn. Res., vol. 12, pp. 2825–2830, 2011.

[13] T. Wolf et al., "Transformers: State-of-the-art natural language processing," in Proc. Conf. Empirical Methods Natural Lang. Process.: Syst. Demonstrations, 2020, pp. 38–45.

[14] J. L. Fleiss, "Measuring nominal scale agreement among many raters," Psychol. Bull., vol. 76, no. 5, pp. 378–382, 1971.

[15] D.-H. Lee, "Pseudo-label: The simple and efficient semi-supervised learning method for deep neural networks," in Proc. ICML Workshop Challenges Representation Learn., 2013.

[16] H. He and E. A. Garcia, "Learning from imbalanced data," IEEE Trans. Knowl. Data Eng., vol. 21, no. 9, pp. 1263–1284, Sep. 2009.

[17] J. R. Landis and G. G. Koch, "The measurement of observer agreement for categorical data," Biometrics, vol. 33, no. 1, pp. 159–174, 1977.

[18] Google Maps, "McDonald's outlet customer reviews," scraped dataset (DATASET.csv), collected 2025–2026.

[19] S. Ramírez, "FastAPI." [Online]. Available: https://fastapi.tiangolo.com

[20] Meta Platforms, "React." [Online]. Available: https://react.dev

Appendix A: Repository and Artefact Structure

The project repository is organised as follows.

- **data/** – DATASET.csv (33,396 raw rows), cleaned_reviews.csv (22,476 rows) and final_pseudo_labeled_dataset.csv (22,448 rows), with a README describing lineage and leakage controls.
- **notebooks/** – 01 Data Cleaning and Preprocessing, 02 Exploratory Data Analysis, 03 Text Preprocessing and Lexical Exploration, 04 Modelling.
- **src/** – data_loader, feature_engineering, models, evaluation_metrics, explainability, cross_validation, error_analysis, efficiency, ablation_studies and robustness_testing modules.
- **labeling/** – annotation guidelines, the 1,000-review pilot sheets, agreement reports (Fleiss' kappa and Krippendorff's alpha), decision records and the labelling scripts.
- **models/** – best_sentiment_model.pkl, linear_svm_model.pkl, logistic_regression_model.pkl, naive_bayes_model.pkl, tfidf_vectorizer.pkl and the distilbert_sentiment directory.
- **results/** – classification_metrics.csv, cross_validation_results.csv, efficiency_benchmark.csv, ablation_results.csv, error_analysis.csv, robustness_results.csv, summary.json and confusion_matrices.
- **Figures/** – the eleven diagnostic figures (01 to 11) used in this report.
- **frontend/** – the React 18 and Vite web application (views, components and the Axios API client).
- **server.py, run_evaluation_pipeline.py, requirements.txt** – the FastAPI server, the master evaluation pipeline and the Python dependencies.
- **reports/** – this thesis, the Final Audit Report, the Data Pipeline Methodology Report and the PDF generator script.

Appendix B: Reproduction Commands and Result Files

The complete study can be reproduced from the project root with the following commands.

Commands

```
pip install -r requirements.txt
python run_evaluation_pipeline.py   # split, train, evaluate, export
python server.py                    # FastAPI backend (port 8000)
cd frontend && npm install && npm run dev   # React UI (5173)
python labeling\scripts\create_pseudo_labeled_dataset.py
python labeling\scripts\compute_iaa.py
```

The metric tables in Chapter 5 are taken from results/classification_metrics.csv and results/cross_validation_results.csv, and the confusion matrices are stored in results/confusion_matrices.

Glossary

Accuracy: The proportion of test reviews whose predicted sentiment matches the true label.

Auto-generated tag: A generic placeholder text that Google inserts when a reviewer gives a star rating without a comment.

Bigram: A sequence of two consecutive words treated as a single text feature.

Brier score: The mean squared difference between predicted class probabilities and the true outcome; lower is better.

Class imbalance: A situation in which some target classes (here, Neutral) are much rarer than others.

Confusion matrix: A table showing, for each true class, how many instances were predicted in each class.

Cross-validation: Repeated training and testing on different partitions of the data to estimate performance and its variability.

DistilBERT: A smaller, faster transformer language model obtained by distilling BERT.

Fine-tuning: Continuing to train a pre-trained language model on a task-specific labelled dataset.

Fleiss' kappa: An agreement statistic for three or more annotators that corrects for chance agreement.

Label leakage: A flaw in which information that reveals the target label is available to the model during training.

Lexicon-based analysis: Scoring text using a pre-built dictionary of words with assigned polarity, without training (for example, VADER).

Log loss: A measure of the quality of predicted probabilities that penalises confident wrong predictions; lower is better.

Macro-F1: The unweighted mean of the F1-scores of all classes; the primary selection metric in this report.

Pseudo-labelling: Assigning labels to unlabelled data using the predictions of a model trained on labelled data.

ROC-AUC, PR-AUC: Areas under the receiver operating characteristic and precision-recall curves; threshold-independent ranking measures.

Sentiment analysis: The automated classification of text as expressing a positive, negative or neutral opinion.

Stratified split: A data split that preserves the class proportions in every subset.

TF-IDF: Term Frequency-Inverse Document Frequency, a weighting that scores terms by their frequency in a document discounted by their frequency across the corpus.

Transformer: A neural network architecture based on self-attention that models context across a whole sequence.

VADER: Valence Aware Dictionary and sEntiment Reasoner, a rule-based sentiment analyser used here as a baseline.